

Vision Inspector

Detailed Computer Vision Project Report

VITyarthi - Build Your Own Project

Field	Value
Course	Computer Vision
Project type	Classical image inspection dashboard
Technology	Python, OpenCV, NumPy, Streamlit, pytest
Version	1.0.0
Student details	To be completed before submission

This report describes the implementation that exists in the repository. It does not claim model training, database persistence, performance benchmarks, or screenshots that were not produced.

1. Introduction

Computer-vision datasets often require a quality check before annotation, training, or publication. Images may be dark, overexposed, blurry, low contrast, dominated by repetitive colors, or lacking useful visual structure. Manual inspection is slow and subjective. Vision Inspector is a local-first dashboard and command-line tool that creates an explainable first-pass inspection report for one PNG or JPEG image.

The project applies classical computer-vision concepts directly: grayscale statistics, color-space conversion, k-means clustering, Canny edge detection, contours, Otsu thresholding, and connected components. The design is intentionally lightweight so that a faculty evaluator can run and demonstrate it on a fresh machine.

2. Problem Statement

Given a local image, a user needs a repeatable way to identify common quality and structure issues before investing time in downstream computer-vision work. The system must provide understandable measurements and warnings without sending the image to a cloud service or making an opaque automated decision.

3. Motivation

- Reduce the time required for first-pass dataset triage.
- Connect syllabus concepts to a working real-world tool.
- Expose reasons behind a quality score instead of hiding them.
- Provide the same analysis engine through both an interactive UI and a CLI.

4. Objectives

- Accept and validate a PNG/JPEG image.
- Calculate quality, color, and structure metrics.
- Return a typed, serializable inspection report.
- Provide useful success, warning, empty, and error states.
- Demonstrate the project with automated tests and a reproducible sample image.

5. Target Users

- Computer-vision students learning image-processing fundamentals.
- Dataset curators performing an initial image audit.
- Developers debugging image-capture or preprocessing pipelines.

6. Scope

In scope are single-image PNG/JPEG loading, validation, quality metrics, dominant-color analysis, structure metrics, dashboard presentation, JSON export, tests, diagrams, and an academic report. Out of scope are object identity recognition, face recognition, model training, cloud storage, persistent user accounts, batch processing, and automatic deletion or modification of source images.

7. Functional Requirements

ID	Requirement	Implementation evidence
FR-1	Accept one PNG/JPEG image.	Streamlit uploader and CLI path argument
FR-2	Validate and decode input.	pipeline.py checks path and cv2.imread result
FR-3	Analyze image quality.	quality.py: brightness, contrast, blur, score, warnings
FR-4	Analyze color properties.	colors.py: HSV colorfulness, mean RGB, k-means palette
FR-5	Analyze visual structure.	structure.py: Canny, contours, Otsu, components
FR-6	Present and export results.	Streamlit cards/tabs and CLI/download JSON

8. Non-Functional Requirements

ID	Requirement	Design response
NFR-1	Performance	Color clustering is bounded to an 80 x 80 thumbnail.
NFR-2	Security and privacy	Images are processed locally; no credentials or network API is required.
NFR-3	Usability	Plain-language labels, tabs, metrics, success, warning, and error states.
NFR-4	Reliability	Unreadable and missing files raise an actionable ValueError.
NFR-5	Maintainability	Single-purpose modules and typed dataclass result contracts.
NFR-6	Resource efficiency	No persistent database and bounded intermediate palette data.

9. Assumptions and Constraints

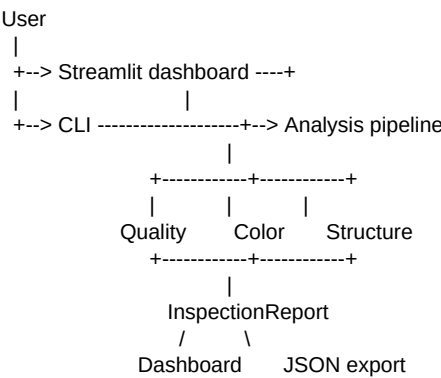
- The input is a valid local image in a format supported by OpenCV.
- The quality score is heuristic and is not a production acceptance guarantee.
- The application analyzes one image per inspection.
- Thresholds are fixed in code for reproducible classroom demonstrations.
- No personal data or authentication is required for the prototype.

10. System Architecture

The system uses a small layered architecture. Presentation adapters are kept separate from domain algorithms. Both the Streamlit application and the CLI call the same pipeline, so the user interface does not duplicate computer-vision logic.

Layer	Responsibility	Files
Presentation	Upload, display, download, and command-line formatting	app.py, vision_inspector/cli.py
Orchestration	Validate input and coordinate modules	vision_inspector/pipeline.py
Domain algorithms	Compute quality, color, and structure measurements	quality.py, colors.py, structure.py
Contracts	Typed immutable outputs and JSON serialization	models.py
Verification	Automated regression and edge-case tests	tests/test_pipeline.py

Architecture diagram



11. Major Modules

Module	Purpose	Key CV concepts
Quality analysis	Find exposure, contrast, and blur signals.	Grayscale mean/std, Laplacian variance
Color analysis	Summarize colorfulness and dominant colors.	BGR to RGB/HSV, saturation, k-means
Structure analysis	Describe edges and foreground-like regions.	Canny, contours, Otsu, connected components
Pipeline/reporting	Provide one stable output contract.	Validation, dataclasses, JSON serialization

12. Detailed Algorithm Design

12.1 Quality analysis

The BGR image is converted to grayscale. Brightness is the grayscale mean, contrast is the grayscale standard deviation, and blur score is the variance of the Laplacian. Warnings are generated for underexposure, overexposure, low contrast, and possible blur. The heuristic score combines exposure, contrast, and sharpness components into a bounded 0-100 value.

12.2 Color analysis

The image is resized to 80 x 80 before clustering to bound cost. OpenCV k-means groups pixels into five color centers using fixed criteria. Centers are converted from BGR to RGB and ordered by pixel frequency. Colorfulness is estimated from average HSV saturation, and mean RGB is calculated from the full image.

12.3 Structure analysis

The grayscale image is processed with Canny edge detection. Edge density is the percentage of nonzero edge pixels. External contours are extracted from the edge map and the largest contour area is normalized by total image area. Otsu thresholding creates a binary image for connected-component counting.

13. Input/Output Contract

Input: a filesystem path or uploaded PNG/JPEG. Output: `InspectionReport(filename, width, height, channels, quality, colors, structure)`. Quality, color, and structure are immutable dataclasses. `to_dict()` supports programmatic use and `to_json()` supports the CLI, download button, and integrations.

14. Storage and Schema Design

The system intentionally uses no persistent database. The source image is not copied permanently and the Streamlit upload is written only to a temporary local file for analysis. The report is held in memory and can be serialized as JSON. The ER diagram in `docs/diagrams/er-diagram.mmd` represents the report object composition rather than database tables.

15. Security Considerations

- No hard-coded secrets, passwords, or API keys.
- No external upload or remote inference service.
- Temporary upload path is used only for the current inspection.
- Source images are not modified or deleted.
- The tool provides advisory metrics, not high-stakes automated decisions.

16. User Workflow

A user selects a PNG/JPEG image, the application validates and decodes it, then runs the three analysis modules. The dashboard displays a summary and module tabs. The user can review warnings, inspect detailed metrics, and download JSON. The CLI follows the same pipeline and prints either a concise summary or formatted JSON.

Workflow diagram

```
START
|
Select image
|
Readable? -- no --> Actionable error --> END
| yes
Run quality + color + structure modules
|
Assemble typed report
|
Review dashboard or export JSON
|
END
```

17. UML Design Artifacts

Use case diagram

```
Dataset curator --> (Upload image)
Dataset curator --> (Review warnings)
Dataset curator --> (Explore colors and structure)
Developer -----> (Run CLI analysis)
Developer -----> (Export JSON)
All use cases -----> Vision Inspector
```

Component/class diagram

```
App -----> Pipeline -----> Quality
|           |-----> Colors
|           |-----> Structure
CLI -----+-----> InspectionReport
              |--> to_dict()
              +--> to_json()
```

Sequence diagram

```
User -> UI/CLI: provide image
UI/CLI -> Pipeline: analyze_image(path)
Pipeline -> Validator: check and decode
Pipeline -> CV modules: run three analyses
CV modules --> Pipeline: typed results
Pipeline -> Report: assemble
Report --> UI/CLI: display or serialize
```

The authoritative Mermaid sources are stored in `docs/diagrams/` and correspond to the implemented modules.

18. Implementation Details

The repository contains a reusable Python package, a Streamlit entry point, a CLI entry point, a deterministic demo-image generator, automated tests, Markdown documentation, Mermaid diagrams, and report builders. The UI uses temporary files for uploads and handles invalid input with a visible error. The CLI shares the same pipeline and therefore has identical measurements.

19. Testing Approach

Test	Purpose	Observed result
Synthetic valid PNG	Validate dimensions and metric bounds.	Passed
Unreadable text file	Validate actionable input error.	Passed
JSON contract	Ensure all modules serialize predictably.	Passed
Missing path	Validate missing-file handling.	Passed
Live Streamlit upload	Verify dashboard workflow and module tabs.	Verified

Automated verification completed with 4 passed tests. No fabricated performance benchmark or accuracy claim is included because the project is a heuristic inspection tool rather than a trained predictive model.

20. Runtime Results

The reproducible demo image `data/sample\_scene.png` produced the following actual CLI report: image size 720 x 480 with 3 channels; quality score 73.83/100; brightness 158.08; contrast 45.81; blur score 435.22; colorfulness 48.95%; mean RGB (163, 162, 117); edge density 1.36%; largest contour ratio 0.6774; connected components 2; no quality warnings.

The live dashboard displayed the uploaded image, summary cards, a no-warning success state, the quality tab, the color tab with palette, the structure tab, and the JSON download control.

21. Design Decisions and Rationale

- Classical CV: transparent and directly aligned with introductory CV concepts.
- Single-image workflow: small enough to demonstrate reliably and easy to understand.
- Typed dataclasses: stable contracts for UI, CLI, tests, and JSON.
- No database: persistence is not needed for a one-image advisory analysis.
- Bounded k-means input: keeps the demonstration responsive and resource efficient.
- Human-readable warnings: preserve interpretability alongside the score.

22. Challenges Faced

- A single score can hide causes, so component metrics remain visible.
- Color clustering can be expensive, so a bounded thumbnail is used.
- The same behavior must be consistent between a browser UI and CLI, so both call one pipeline.

23. Learnings and Key Takeaways

- Classical CV primitives can support practical dataset triage without model training.
- Separating algorithms from presentation improves testability and reuse.
- A measurement plus an explanation is more useful than an unexplained score.
- Reproducible sample data helps faculty demonstrate the project quickly.

24. Future Enhancements

- Batch directory analysis with CSV/JSONL export.
- Configurable thresholds and saved inspection profiles.
- Perceptual-hash duplicate detection.
- Interactive plots and before/after enhancement previews.
- Optional ML detector as a separate, clearly labeled module.

25. References

- VITyarthi, Build Your Own Project: General Project Instructions and Submission Guidelines, provided assignment PDF.
- OpenCV documentation for image processing, color conversion, k-means, Canny, contours, thresholding, and connected components.
- NumPy documentation for array statistics and reshaping.
- Streamlit documentation for file upload, tabs, metrics, and download buttons.

26. Submission Checklist

Deliverable	Location
Source code	app.py, vision_inspector/, scripts/
Automated tests	tests/test_pipeline.py
README and statement	README.md, statement.md

Deliverable	Location
Design documentation	docs/*.md
Diagrams	docs/diagrams/*.mmd
Report	This PDF and Vision_Inspector_Project_Report.pdf
Demo image	data/sample_scene.png

Manual fields before portal submission: student name, registration/roll number, faculty name, semester, academic year, and any faculty-specific cover-page format. Runtime screenshots should be saved in docs/screenshots if the submission portal requires image files.